



PRESIDENCY COLLEGE

(AUTONOMOUS)

AFFILIATED TO BENGALURU CITY UNIVERSITY, APPROVED BY AICTE, DELHI & RECOGNISED BY THE GOVT. OF KARNATAKA

RE-ACCREDITED BY NAAC WITH 'A' GRADE

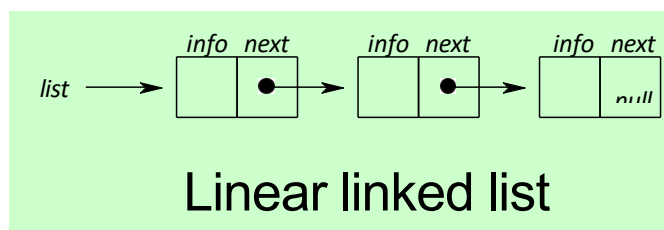
Unit 2: Linked Lists

Notes Prepared by Prof. Veena S Badiger

Introduction to linked lists Implementation of linked lists Applications of linked lists Implementing a dynamic array Implementing a hash table Implementing a graph using an adjacency list Time and space complexity analysis of linked list operations Implement a linked list Implement a dynamic array using a linked list Implement a hash table using a linked list

Linked List

- A **linked list** is a linear collection of data elements, called **nodes**, where the linear order is given by means of **pointers**.
- Each **node** is divided into two parts:
 - The first part contains the **information** of the element and
 - The second part contains the address of the next node (**link / next pointer field**) in the list.

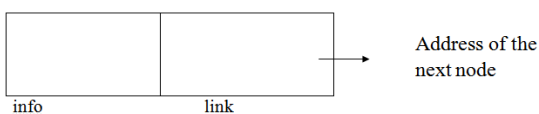


Components of linked list

A singly linked list is a linear collection of data elements called nodes where each node is divided into two parts:

1. Information field
2. Pointer to the next node/link field

Node = info + link



- Information field holds the actual value
- Pointer or link field contains the address of the next node,
- Last node of the list contains NULL(‘0’) in the link indicating that it is end of list.

- **Null pointer-** link field/ pointer field of the last node contains a zero
- **External pointer-** it is a pointer to the very first node in the linked list.

Representation of linked list

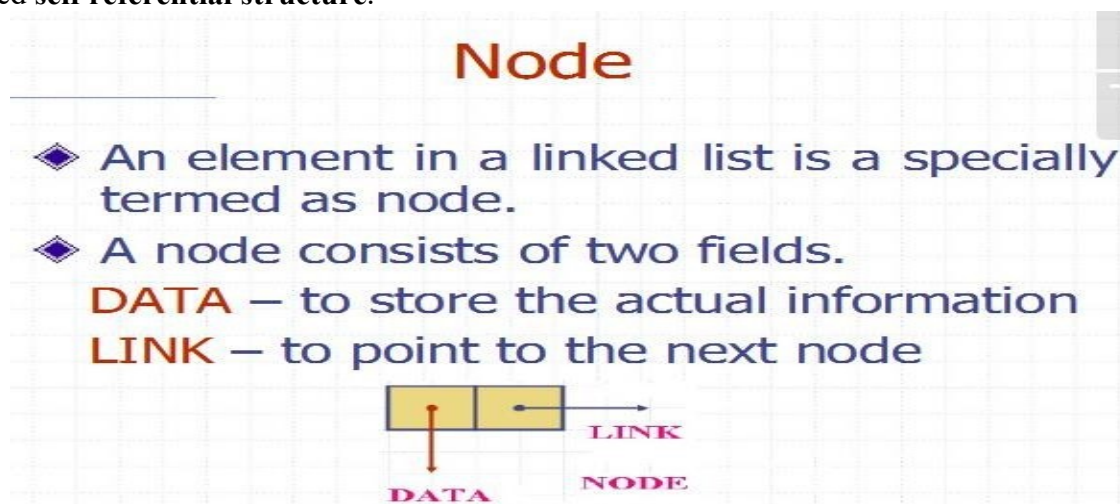
- In order to create a linked list a structure is to be declared that defines all the list entries.

Syntax-

- **struct node**

```
{
    int info;
    struct node *link;
};
```
- link pointer points to the next node of same structure type.

When node point to same structure type is called **self referential structure**.



What are the Uses of Linked List?

- The list is not required to be contiguously present in the memory. The node can exists any where in the memory and linked together to make a list. This achieves **optimized utilization of space**.
- Memory size need not to be declared in advance.
- Empty node can not be present in the linked list.
- We can store values of primitive types or objects in the singly linked list.

Array	Linked list
An array is a homogeneous collection of elements of a similar data type.	A linked list is a linear collection of data elements, called nodes . Each node consists of two parts, i.e., data and address.

Array elements store in a contiguous memory location.	Linked list elements can be stored anywhere in the memory or randomly stored.
Array works with a static memory. Static means the size is fixed and cannot be changed at the run time.	Linked list works with dynamic memory. Dynamic memory means that the memory size can be changed at the run time according to our requirements.
Array elements are independent of each other.	Linked list elements are dependent on each other. Each node contains the address of the next node.

Array takes more time while performing any operation like insertion, deletion, etc.	Linked list takes less time while performing any operation like insertion, deletion, etc.
Accessing any element in an array is faster as the element in an array can be directly accessed through the index.	Accessing an element in a linked list is slower as it starts traversing from the first element of the linked list.
memory is allocated at compile-time.	memory is allocated at run-time.
Memory utilization is inefficient in the array. For example, if the size of the array is 6, and array consists of 3 elements only then the rest of the space will be unused.	Memory utilization is efficient in the case of a linked list as the memory can be allocated or deallocated at the run time according to our requirement.

Memory Representation of LL

◆ Two ways

- 1) Static representation using array
- 2) Dynamic representation using free pool of storage.

◆ In Static representation of a single linked list, two arrays are maintained:

- 1) Array of Data
- 2) Array of Links

Dynamic Representation

- ◆ The efficient way is using free pool of storage.
- ◆ There is a **memory bank (free memory spaces)** and **memory manager (program)**.
- ◆ During the creation of a linked list, whenever a node is required the request is placed to the memory manager.
- ◆ Then it searches the memory bank for the block requested and if found, grants the desired block to the caller.

Advantages (advantages of linked list over array)

1. Efficient memory utilization.

Memory of the linked list is not preallocated. Memory can be allocated whenever required. It is deallocated when it is no longer needed.

2. Insertion and deletion operations are easier and efficient.

Linked list provides flexibility in inserting a data item at a specified position and deletion of a data item from the given position.

3. No wastage of memory.

As the linked list dynamic data structures it can grow when needed and shrink when it is no longer needed. Thus no wastage of memory.

4. Extensive manipulations.

Any number of complex operations can be performed without the prior idea about the memory space available. There will be no overflow and underflow conditions during insertion and deletion.

5. Arbitrary memory locations.

In linked list the memory locations need not be consecutive. They may be any arbitrary positions. The accessing of the elements is easier as every location contains data and address to the next location. They are ordered not by physical location in memory but by their logical links stored as part of the data with node itself.

What is dynamic memory allocation?

- The process of allocating memory at runtime is known as **dynamic memory allocation**.
- The functions `malloc()` and `calloc()` are used to allocate memory and `free()` is used to deallocate memory.

Types of Linked List

1. Singly linked list
2. Circular linked list

3. Doubly linked list
4. Header linked list
5. Circular doubly linked list

SINGLY LINKED LIST

A singly linked list or one-way list is a linear collection of data elements called nodes, where each node is divided into 2 parts, info field and link field.

The link field of each node will point to the next node and link field of last node always point to NULL.

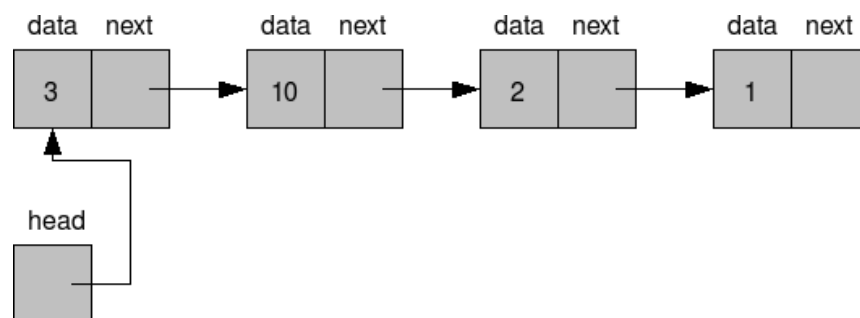
Start is a pointer, which points to the first node of the linked list.

ADVANTAGE :-

- Insertions and Deletions can be done easily.
- It does not need movement of elements for insertion and deletion.
- Its space is not wasted as we can get space according to our requirements.
- Its size is not fixed.
- It can be extended or reduced according to requirements.

DISADVANTAGE :-

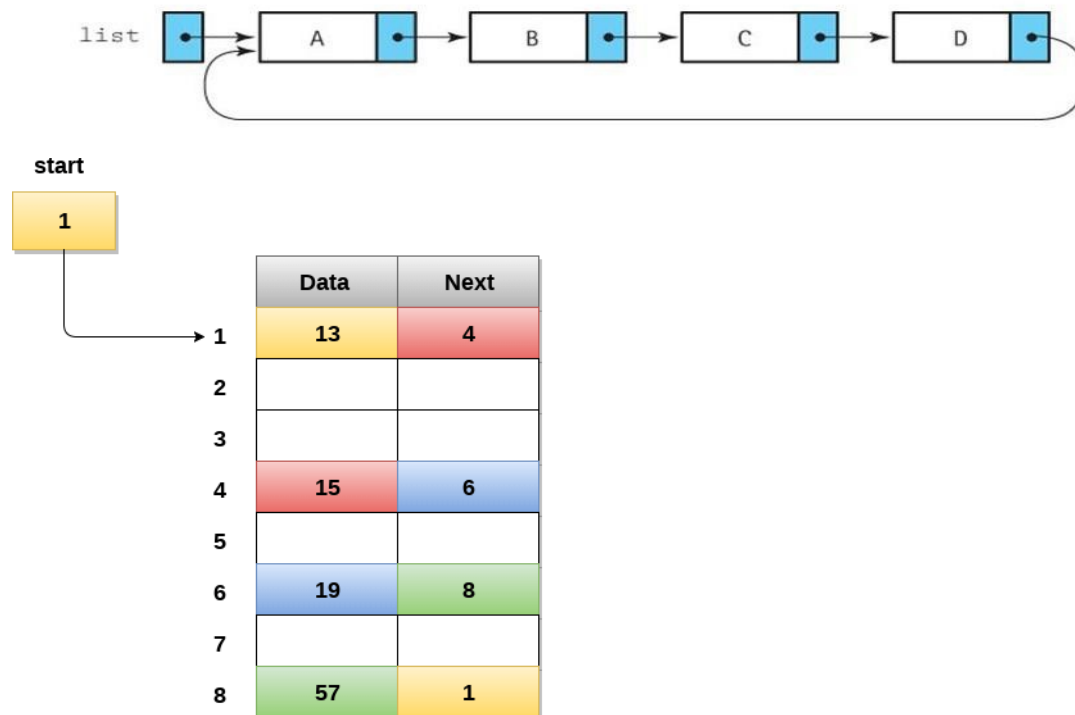
- It requires more space as pointers are also stored with information.
- Different amount of time is required to access each element.
- If we have to go to a particular element then we have to go through all those elements that come before that element.
- we can not traverse it from last & only from the beginning.
- It is not easy to sort the elements stored in the linear linked list.



CIRCULAR LINKED LIST

- In **Circular Linked List**, the list element are arranged in the same way as in the singly linked list having only one difference that the last element of the circular linked list always point to the first node of the list i.e. it means that last node does not point to NULL.
- In other words, we can say that the circular list is a list which does not have an end.

Circular linked list A list in which every node has a successor; the “last” element is succeeded by the “first” element



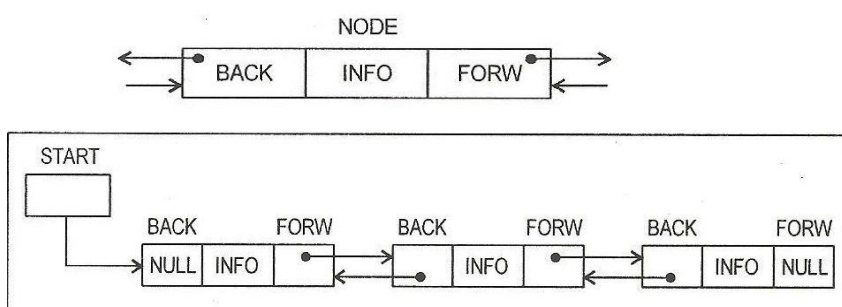
Memory Representation of a circular linked list

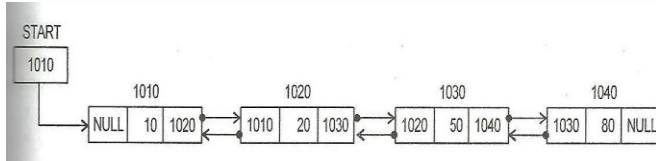
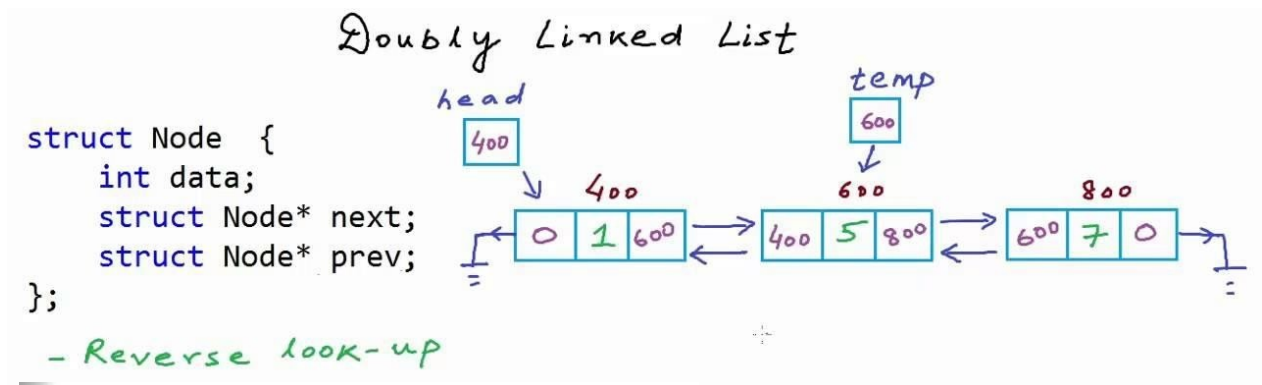
Advantages of circular list:

1. The circular list can be traversed to any node from any node.
2. A node can be inserted at or deleted from any position of the linked list.
3. All nodes link address will have valid addresses, instead of NULL pointer.
4. We can traverse the whole list starting from any point.

DOUBLY LINKED LIST

- In **Doubly Linked List** the node of the list generally consists of the three fields i.e. one num field which contains the data value, and two pointer fields ie. next pointer(FORW) and previous pointer(BACK).
- next pointer contains the address of the next element in the list, and previous pointer would be containing the address of the previous node in the list.





Advantages:

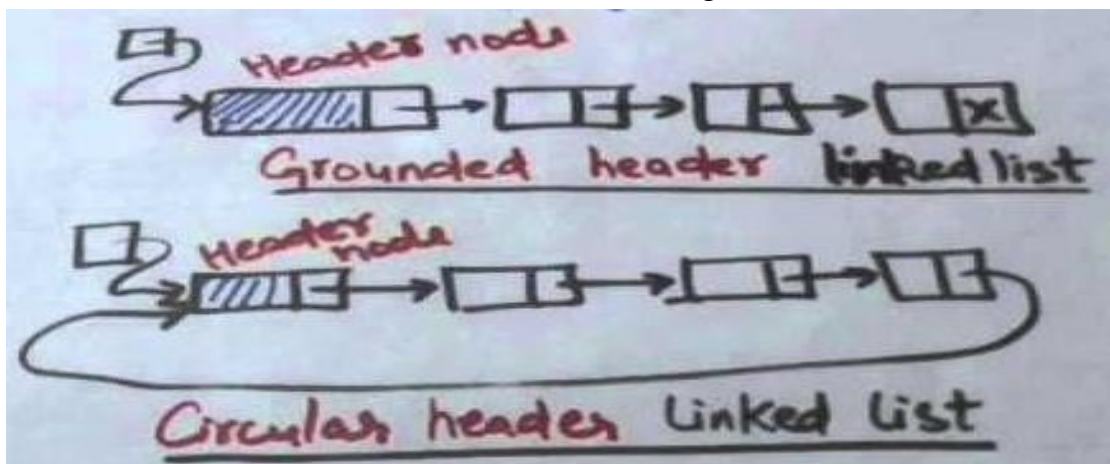
1. Can be traversed in forward and backward direction.
2. If we know particular node address, successor node address and predecessor node address, insertion and deletion is easy.
3. Is used to represent the tree effectively.

Disadvantage:

1. Extra memory is required to store the back pointer.

HEADER LINKED LIST

- A header linked list which always contains a special node, called the header node, at the beginning of the list. **The following are two kinds of widely used header list:**
- A **grounded header list** is a header list where the last node contains the null pointer.
- A **circular header list** is a header list where the last node points back to the header node. The header node also acts as a sentinel indicating the end of the list.



What is a NULL pointer, external pointer and empty list in context to the linked list ?

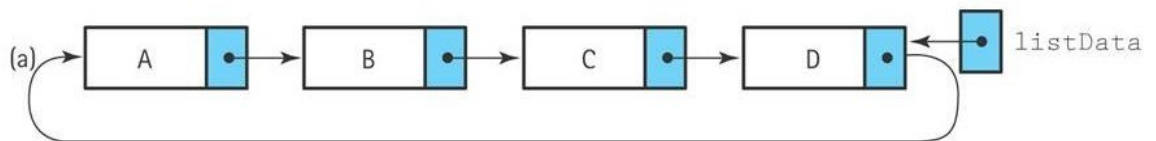
A null pointer has a reserved value which does not point to any valid function.

- It is used to Initialize pointers and also represents end of a list.

Ex: ptr=NULL

External pointer is a pointer that points to the very first node in the linked list, it enables us to access the entire linked list.

External Pointer to the Last Node



An empty list is a list that contains no nodes in the list, it is also called a null list. The value of external pointer will be zero for an empty list. A list can be made an empty list by assigning a NULL value to the external pointer. i.e.

- **head = NULL;** where head is an external pointer.

Empty List

Empty Linked list is a single pointer having the value of NULL.

```
head = NULL;
```



Length of a List int

```
len = 0;
```

```
while(curptr != NULL)
```

```
{
```

```
    len++;
```

```
    curptr = curptr->link;
```

```
}
```

```
return (len);
```

Explain the applications of LL.

1. Implementation of stacks and queues.

2. Implementation of graphs : Adjacency list representation of graphs is most popular which uses linked list to store adjacent vertices.
3. Dynamic memory allocation : We use linked list of free blocks.
4. Maintaining directory of names.
5. Performing arithmetic operations on long integers.
6. Manipulation of polynomials by storing constants in the node of linked list
7. representing sparse matrices

Disadvantages of Linked Lists

- The memory is wasted as pointers require extra memory for storage.
- No element can be accessed randomly; it has to access each node sequentially.
- Reverse Traversing is difficult in linked list

What are the disadvantages of linked list?

Wastage of Memory –

Pointer Requires extra memory for storage.

No Random Access

In array we can access nth element easily just by using $a[n]$.

In Linked list no random access is given to user, we have to access each node sequentially.

Time Complexity

Array can be randomly accessed, while the Linked list cannot be accessed Randomly

Individual nodes are not stored in the contiguous memory Locations.

Access time for Individual Element is $O(n)$ whereas in Array it is $O(1)$.

Reverse Traversing is difficult

In case if we are using singly linked list then it is very difficult to traverse linked list from end.

If using doubly linked list then though it becomes easier to traverse from end but still it increases again storage space for back

Heap Space Restriction

Whenever memory is dynamically allocated, It utilizes memory from heap.

Memory is allocated to Linked List at run time if and only if there is space available in heap. If there is insufficient space in heap then it won't create any memory.

What are the Operations supported by a linked list

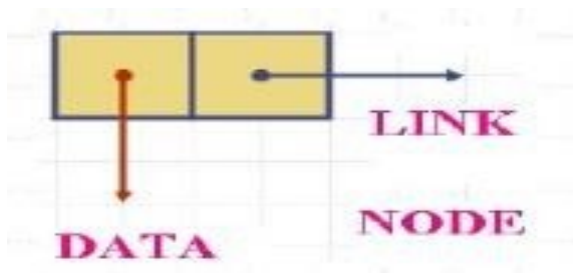
- **Insertion** – Adds an element at the beginning of the list.
- **Deletion** – Deletes an element at the beginning of the list.
- **Insert Last** – Adds an element at the end of the list.
- **Delete Last** – Deletes an element from the end of the list.
- **Insert After** – Adds an element after an item of the list.
- **Delete** – Deletes an element from the list using the key.
- **Display forward** – Displays the complete list in a forward manner.
- **Display backward** – Displays the complete list in a backward manner.

Explain the node structure of a SLL.

In dynamic representation of LL, a node is created dynamically whenever it is required. Any no of nodes can be created depending upon the requirement.

```
struct node
{
    int info;
    struct node *link;
};
```

“link” is the pointer that points to the next node of the same structure type.



A node may contain any number of information fields.

```
struct node
{
```

```
int info1, info2, info3, .....info6; struct node
*link;
};
```

Write an algorithm for Traversing in singly linked list 5m

Traversing is the most common operation that is performed in almost every scenario of singly linked list.

Traversing means visiting each node of the list once in order to perform some operation on that. This will be done by using the following algorithm.

Algorithm Traversing STEP 1:

SET PTR = HEAD STEP 2: IF

PTR = NULL

WRITE "EMPTY LIST"

GOTO STEP 7

END OF IF

STEP 3: REPEAT STEP 4 AND 5 UNTIL PTR != NULL

STEP 4: PRINT PTR → DATA

STEP 5: PTR = PTR → NEXT
[END OF LOOP]

STEP 6: EXIT

Write an algorithm for Searching element in singly linked list.

Searching is performed in order to find the location of a particular element in the list. Searching any element in the list needs traversing through the list and make the comparison of every element of the list with the specified element. If the element is matched with any of the list element then the location of the element is returned from the function.

Algorithm SEARCH

Step 1: SET PTR = HEAD

Step 2: Set i = 0

STEP 3: IF PTR = NULL

WRITE "EMPTY LIST"

GOTO STEP 8

END OF IF

STEP 4: REPEAT STEP 5 TO 7 UNTIL PTR != NULL

STEP 5: if ptr → data = item

write i+1

End of IF

STEP 6: I = I + 1

STEP 7: PTR = PTR → NEXT [END OF LOOP]

STEP 8: EXIT

Write an algorithm to Insert an element at the beginning in singly linked list 5m

Or

Write the steps to insert an element at the beginning. 5m/6m

- Inserting a new element into a singly linked list at beginning is quite simple.

Following are the steps

1. Allocate the space for the new node and store data into the data part of the node. This will be done by the following statements.

```
ptr = (struct node *) malloc(sizeof(struct node *)); ptr → data =  
item
```

2. Make the link part of the new node pointing to the existing first node of the list. This will be done by using the following statement.

```
ptr->next = head;
```

3. At the last, we need to make the new node as the first node of the list this will be done by using the following statement.

head = ptr;

Write an algorithm to Insert an element at the beginning in singly linked list 5m

Algorithm INSERT_BEG

Step 1: IF PTR = NULL

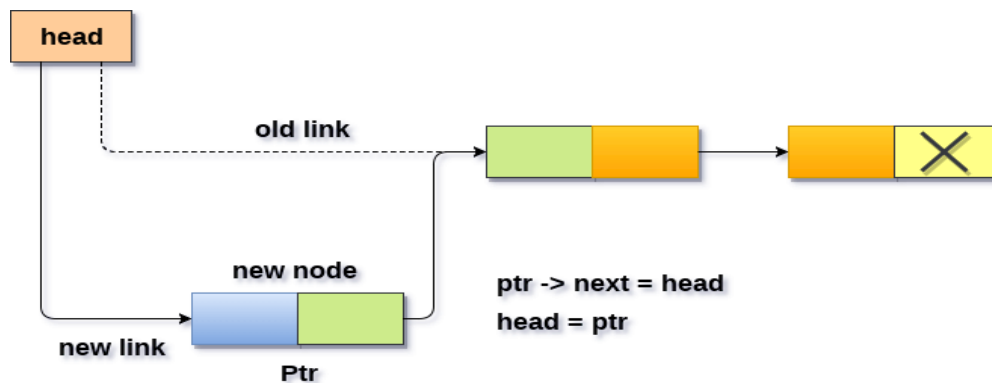
Write OVERFLOW
Go to Step 7 [END
OF IF]

Step 2: SET NEW_NODE = PTR

Step 3: SET PTR = PTR → NEXT

Step 4: SET NEW_NODE → DATA = VAL **Step 5:**
SET NEW_NODE → NEXT = HEAD **Step 6:** SET
HEAD = NEW_NODE

Step 7: EXIT

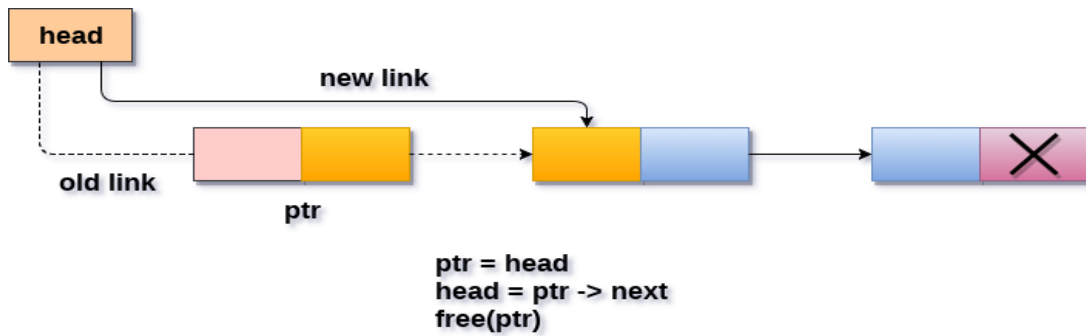


Write an algorithm to Delete an element from singly linked list at beginning 5m

Deleting a node from the beginning of the list is the simplest operation of all. It just need a few adjustments in the node pointers. Since the first node of the list is to be deleted, therefore, we just need to make the head, point to the next of the head. This will be done by using the following statements.

Steps needed for deletion at the beginning of the Singly Linked List

1. Move the current head from 1st node to the next node.
2. Delete the first node using the free method.
3. If the Linked List is empty that it is not possible to delete.



Deleting a node from the beginning

Circular linked list program using C

```

#include <stdio.h>
#include <stdlib.h>

```

```

// Node structure

```

```

struct Node {
    int data;
    struct Node* next;
};

```

```

struct Node *head=NULL;

```

```

// Function to create a new node

```

```

struct Node* createNode(int data)

```

```

{
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

```

```

// Function to insert a node at the beginning

```

```

void insertAtBeginning( int data)

```

```

{
    struct Node * temp = head;
    struct Node * newNode = createNode(data);
    if (head == NULL) {
        head = newNode;
        newNode->next = head;
    }
    else {

```



```

        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
        head = newNode;
    }
}

```

// Function to insert a node at the end

```

void insertAtEnd(int data)
{
    struct Node* newNode = createNode(data);
    if ( head == NULL) {
        head = newNode;
        newNode->next = head;
    }
    else {
        struct Node * temp = head;
        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
    }
}

```

// Function to insert a node at a specific position

```

void insertAtPosition(int data, int position)
{
    struct Node * newNode = createNode(data);
    if (head == NULL && position == 0) {
        head = newNode;
        newNode->next = head;
    }
    else if (position == 0) {
        insertAtBeginning(head, data);
    }
    else {
        struct Node * temp = head;
        int i = 0;
        while (i < position - 1) {
            temp = temp->next;

```

```

        i++;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}
}

```

// Function to delete a node from the beginning

void deleteFromBeginning()

```

{
    if (head == NULL) {
        return;
    }
    else if ((head)->next == *head) {
        free(head);
        head = NULL;
    }
    else {
        struct Node * temp = head;
        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = head->next;
        struct Node *toDelete = head;
        head = head->next;
        free(toDelete);
    }
}

```

// Function to delete a node from the end

void deleteFromEnd()

```

{
    if (head == NULL) {
        return;
    }
    else if (head->next == head) {
        free(head);
        head = NULL;
    }
    else {
        struct Node *secondLast = head;
        while (secondLast->next != head) {
            secondLast = secondLast->next;
        }
    }
}

```

```

        struct Node * last = secondLast->next;
        secondLast->next = head;
        free(last);
    }
}

```

// Function to delete a node from a specific position

```

void deleteAtPosition(int position)
{
    if (head == NULL) {
        return;
    }
    else if (position == 0) {
        deleteFromBeginning(head);
    }
    else {
        struct Node *temp = head;
        int i = 0;
        while (i < position - 1) {
            temp = temp->next;
            i++;
        }
        struct Node *toDelete = temp->next;
        temp->next = temp->next->next;
        free(toDelete);
    }
}

```

// Function to traverse and print the circular linked list

```

void traverse()
{
    if (head == NULL) {
        return;
    }
    struct Node *temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("HEAD\n");
}

```

// Function to search for a node with a given key

```

int search( int key)

```

```

{
    if (head == NULL) {
        return 0;
    }
    struct Node *temp = head;
    do {
        if (temp->data == key) {
            return 1; // Key found
        }
        temp = temp->next;
    } while (temp != head);
    return 0; // Key not found
}

```

// Driver program

int main()

```

{
    struct Node *head = NULL;

    // Insertion
    insertAtEnd(10);
    insertAtEnd(20);
    insertAtBeginning( 5);
    insertAtPosition( 15, 2);

    // Traversal
    printf("Circular Linked List: ");
    traverse();

    // Deletion
    deleteFromEnd();
    deleteAtPosition(1);

    // Traversal after deletion
    printf("Circular Linked List after deletion: ");
    traverse();

    // Searching
    int key = 10;
    if (search( key)) {
        printf("Element %d is found in the linked list.\n", key);
    }
    else {
        printf("Element %d is not found in the linked list.\n", key);
    }
}

```

```

    }

    return 0;
}

```

Doubly linked list program using C

```

#include <stdio.h>
#include <stdlib.h>
// node creation
struct Node {
    int data;
    struct Node * next;
    struct Node* prev;};
struct Node *head=NULL;

// insert node at the front
void insertFront(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = head;
    newNode->prev = NULL;
    if (head != NULL)
        head->prev = newNode;
    head = newNode;
}

// insert a node after a specific node
void insertAfter(struct Node* prev_node,int data) {
    if (prev_node == NULL) {
        printf("previous node cannot be null");
        return;
    }
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = prev_node->next;
    prev_node->next = newNode;
    newNode->prev = prev_node;
    if (newNode->next != NULL)
        newNode->next->prev = newNode;
}

// insert a newNode at the end of the list
void insertEnd( int data) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;

```

```
newNode->next = NULL;
```

```
struct Node *temp = *head;  
if (head == NULL) {  
    newNode->prev = NULL;  
    head = newNode;  
}
```

```
while (temp->next != NULL)  
    temp = temp->next;
```

```
temp->next = newNode;  
newNode->prev = temp;  
}
```

```
// delete a node from the doubly linked list
```

```
void deleteNode(struct Node *del_node) {  
    if (head == NULL || del_node == NULL)  
        printf(" No nodes to delete");  
    if (head == del_node)  
        head = del_node->next;  
  
    if (del_node->next != NULL)  
        del_node->next->prev = del_node->prev;  
  
    if (del_node->prev != NULL)  
        del_node->prev->next = del_node->next;  
    free(del_node);  
}
```

```
// print the doubly linked list
```

```
void displayList(struct Node* node) {  
    struct Node *last;  
    while (node != NULL) {  
        printf("%d->", node->data);  
        last = node;  
        node = node->next;  
    }  
    if (node == NULL)  
        printf("NULL\n");  
}
```

```
int main() {  
    insertEnd(5);
```

```

insertFront( 1);
insertFront( 6);
insertEnd( 9);
insertAfter( 11);
insertAfter(head->next, 15);

displayList(head);

deleteNode(head->next->next->next->next->next);
displayList(head);
}

```

HASHING:

- Hashing is the process of transforming a string of characters into a short integer value (usually called key). This short integer value is used as index to get the original string.
- It is used to retrieve items in a database faster
- It is an efficient way of implementing dictionaries by performing the following operations.:
 - ♦ Searching: To search for an item in the given set. Each entry in the set may have many fields such as name, id, roll_no etc.
 - ♦ Insertion: An item can be inserted into a given set
 - ♦ Deletion: An item can be deleted from the set

Hash function, Hash value, Hash table

A **hash function** is a function that takes an input (or ‘message’) and returns a fixed-size string of bytes. The output is a number, is called the **hash code or hash value**.

The table on which insertion, deletion and retrieve operations takes place after computing the hash value is called **hash table**.

The size of hash table is limited and so it is necessary to map the given data into this fairly restricted set of integers.

Example 1: Create the hash table for the items 15, 11, 22, 14 and 33 where size of table is 5. Assume the hash function = $\text{item \% size_of_table}$.

Solution: Let us compute the hash value for each item.

Item = 15. Hash value = $15 \% 5 = 0$. So, insert 15 into $a[0]$
 Item = 11. Hash value = $11 \% 5 = 1$. So, insert 11 into $a[1]$
 Item = 22. Hash value = $22 \% 5 = 2$. So, insert 22 into $a[2]$
 Item = 14. Hash value = $14 \% 5 = 4$. So, insert 14 into $a[4]$
 Item = 33. Hash value = $33 \% 5 = 3$. So, insert 33 into $a[3]$

Inserting these values at the specified position, the contents of hash table are shown below:

	0	1	2	3	4
a	15	11	22	33	14

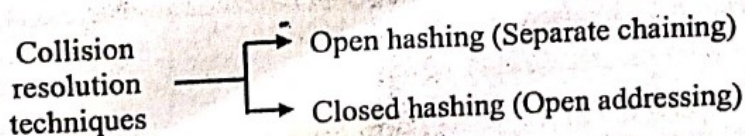
Hash table

For reference.....

Now, suppose we want to insert an item 37 into above table. Its hash value = $37 \% 5 = 2$ i.e., item 37 has to be inserted into $a[2]$. But, there is an item already in that location. This results in collision. We say that the second item is hashed into value which already exists. This is called collision.

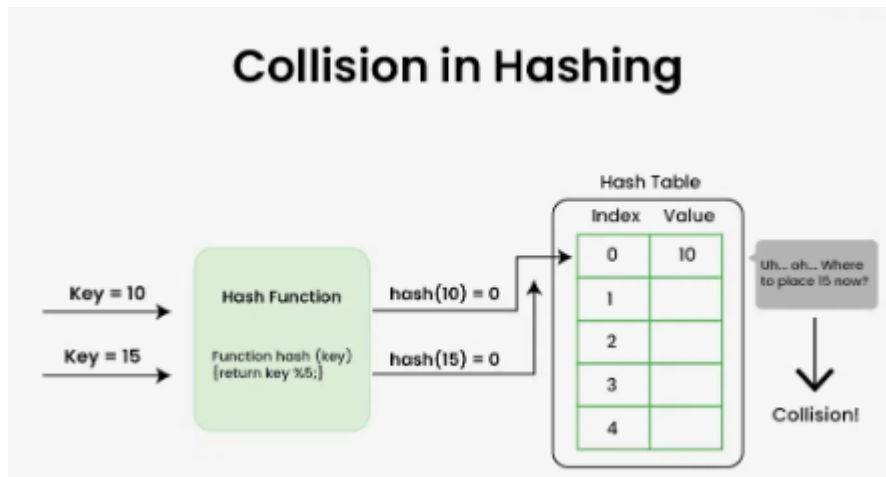
Note: We can expect collision even if the number of keys generated is less than the size of the hash table. Most of the time collision can not be avoided and in the worst case all keys may have the same value.

Now, the question is "How to avoid collision?" The various hashing techniques using which collision can be avoided (collision resolution techniques) are shown below:



WHAT IS COLLISION IN HASHING?

- In **Hashing**, hash functions were used to generate hash values.
- The hash value is used to create an index for the keys in the hash table.
- The hash function may return the same hash value for two or more keys.
- When two or more keys have the same hash value, a **collision** happens.
- To handle this collision, we use **Collision Resolution Techniques**.



Collision Resolution Techniques

There are mainly two methods to handle collision:

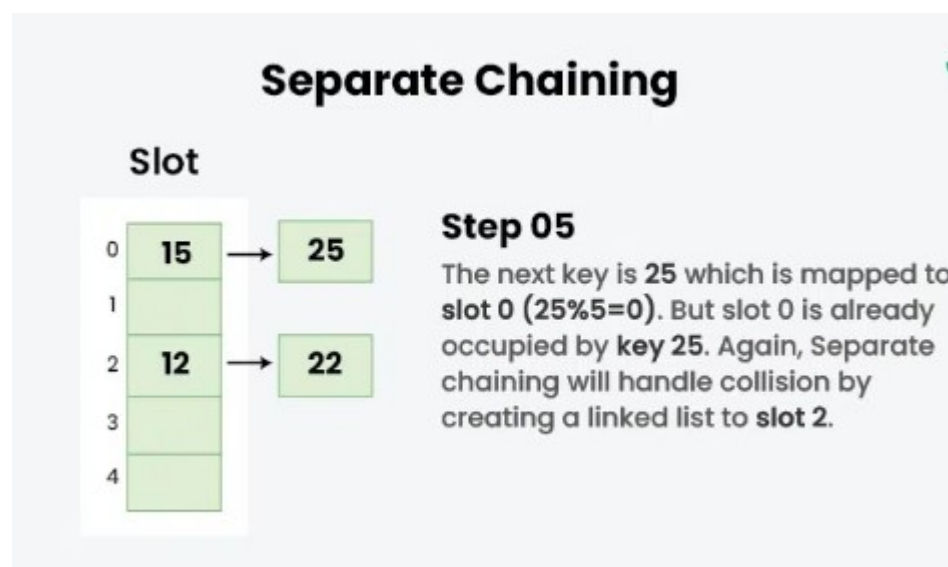
1. **Separate Chaining or open Hashing**
2. **Open Addressing or Closed Hashing**

1) Separate Chaining or open Hashing

In this technique, we make each cell of the hash table point to a linked list of records that have the same hash function value. Chaining is simple but requires additional memory outside the table.

Example1: for a given a hash function and we have to insert some elements in the hash table using a separate chaining method.. *Elements = 12, 15, 22, 25 and 37.*

Hash function = key % 5,

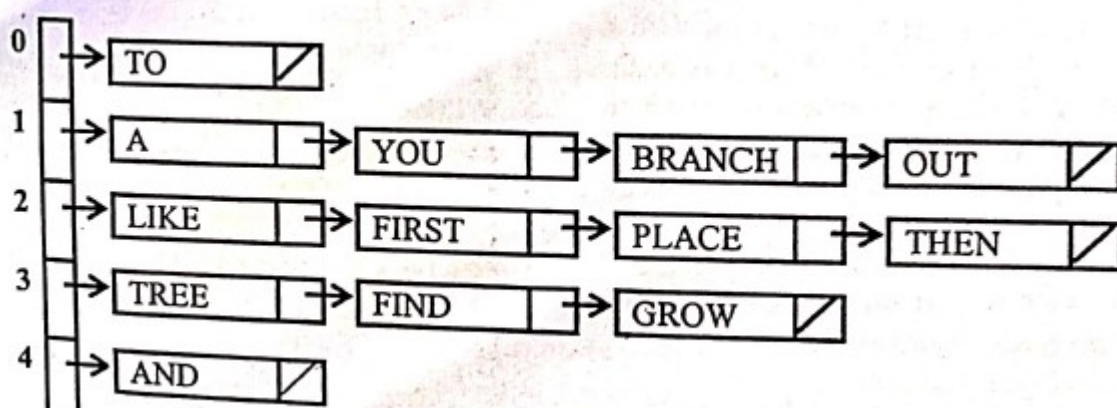


EXAMPLE2 – using separate chaining

Example 2: Let the size of hash table is 5. Insert the following words LIKE, A, TREE, YOU, FIRST, FIND, A, PLACE, TO, GROW, AND, THEN, BRANCH, OUT into a hash table using appropriate hash function and show how each item is inserted into the table, how an item can be searched and deleted

REFER CLASS NOTES FOR SOLUTION

LIKE	$12+9+11+5 = 37$	2
A	$1 = 1$	1
TREE	$20+18+5+5 = 48$	3
YOU	$25+15+21 = 61$	1
FIRST	$6+9+18+19+20 = 72$	2
FIND	$6+9+14+4 = 33$	3
A	$1 = 1$	1
PLACE	$16+12+1+3+5 = 37$	2
TO	$20+15 = 35$	0
GROW	$7+18+15+23 = 63$	3
AND	$1+14+4 = 19$	4
THEN	$20+8+5+14 = 47$	2
BRANCH	$2+18+1+14+3+8 = 46$	1
OUT	$15+21+20 = 56$	1



2) Open Addressing or Closed Hashing

- In [open addressing](#), all elements are stored in the hash table itself.
- Each table entry contains either a record or NIL.

- When searching for an element, we examine the table slots one by one until the desired element is found or it is clear that the element is not in the table.

Following are the strategies

a) Linear Probing

b) Double Hashing

a) Linear Probing

- In this technique, if an item is to be inserted its hash value is computed.
- If the position identified by hash value is empty, insert the item in that location.
- If item already exists, then in the next empty location item is inserted.
- This way of avoiding the collision is called linear probing.

EXAMPLE2 – using Linear probing

Example 3: Let the size of hash table is 15. Insert the following words LIKE, A, TREE, YOU, FIRST, FIND, A, PLACE, TO, GROW, AND, THEN, BRANCH, OUT into a hash table using appropriate hash function and show how each item is inserted into the table, how an item can be searched and deleted using linear probing

Refer class notes for solution

Words	Sum of positions	hash value sum %15
LIKE	12+9+11+5 =37	7
A	1 =1	1
TREE	20+18+5+5 =48	3
YOU	25+15+21 =61	1
FIRST	6+9+18+19+20 =72	12
FIND	6+9+14+4 =33	3
A	1 =1	1
PLACE	16+12+1+3+5 = 37	7
TO	20+15 =35	5
GROW	7+18+15+23 =63	3
AND	1+14+4 = 19	4
THEN	20+8+5+14 =47	2
BRANCH	2+18+1+14+3+8 =46	1
OUT	15+21+20 =56	11

Item = LIKE.	It is inserted into position 7
Item = A.	It is inserted into position 1
Item = TREE	It is inserted into position 3
Item = YOU	Its hash value is 1 and that location is full. So, it is inserted in the \
	next available location 2 as shown in table 7.6.b.
Item = FIRST	It is inserted into position 12
Item = FIND	Its hash value is 3. It is already present and has to be inserted into
	next subsequent empty location.

Thus, keep inserting all the items into the table. The complete hash table using linear probing is shown below:

0	
1	A
2	
3	TREE
4	
5	
6	
7	LIKE
8	
9	
10	
11	
12	
13	
14	

(a)

0	
1	A
2	YOU
3	TREE
4	FIND
5	
6	
7	LIKE
8	PLACE
9	
10	
11	
12	FIRST
13	
14	

(b)

0	
1	A
2	YOU
3	TREE
4	FIND
5	TO
6	GROW
7	LIKE
8	PLACE
9	AND
10	THEN
11	BRANCH
12	FIRST
13	OUT
14	

(c)

Table 7.6 To insert various items into hash table

Example 2 (LINEAR PROBING): Let us consider a simple hash function as “key mod 5” and a sequence of keys that are to be inserted are 50, 70, 76, 85, 93.

Slot

0
1
2
3
4

Step 01

Empty hash table with range of hash values from 0 to 4 according to the hash function provided.

Slot

0	50
1	
2	
3	
4	

Step 02

The first key to be inserted is **50** which is mapped to **slot 0** ($50\%5=0$)

Slot

0	50
1	70
2	
3	
4	

Step 03

The next key is **70** which is mapped to **slot 0** ($70\%5=0$) but **50** is already at **slot 0** so, search for the next empty slot and insert it.

Slot

0	50
1	70
2	76
3	
4	

Step 04

The next key is **76** which is mapped to **slot 1** ($76\%5=1$) but **70** is already at **slot 1** so, search for the next empty slot and insert it.

Slot

0	50
1	70
2	76
3	85
4	

Step 05

The next key is **85** which is mapped to **slot 0** ($85\%5=0$), but **50** is already at **slot number 0** so, search for the next empty slot and insert it. So insert it into **slot number 3**.

Slot	
0	50
1	70
2	76
3	85
4	93

Step 06
 The next key is **93** which is mapped to **slot 3** ($93\%5=3$), but **85** is already at **slot 3** so, search for the next empty slot and insert it. So insert it into **slot number 4**.

b) Double hashing

- Double hashing is a collision resolution technique used in hash tables.
- It works by using two hash functions to compute two different hash values for a given key.
- The first hash function is used to compute the initial hash value, and the second hash function is used to compute the step size for the probing sequence.

Advantages:

- Double hashing has the ability to have a low collision rate.

Disadvantages:

- it requires the use of two hash functions, which can increase the computational complexity of the insertion and search operations.

Double hashing can be done using :

$(hash1(key) + i * hash2(key)) \% TABLE_SIZE$

Here $hash1()$ and $hash2()$ are hash functions and $TABLE_SIZE$ is size of hash table.

(We repeat by increasing i when collision occurs)

20 34 45 70 50

$H1 = key \% 11$

$H2 = 8 - (key \% 8)$

Implement a hash table using a linked list using C language

```
#include<stdio.h>
#include<stdlib.h>
#define size 7
```

```
struct node
```

```

{
    int data;
    struct node *next;
};

struct node *chain[size];

void init()
{
    int i;
    for(i = 0; i < size; i++)
        chain[i] = NULL;
}

void insert(int value)
{
    //create a newnode with value
    struct node *newNode = malloc(sizeof(struct node));
    newNode->data = value;
    newNode->next = NULL;

    //calculate hash key
    int key = value % size;

    //check if chain[key] is empty
    if(chain[key] == NULL)
        chain[key] = newNode;
    //collision
    else
    {
        //add the node at the end of chain[key].
        struct node *temp = chain[key];
        while(temp->next)
        {
            temp = temp->next;
        }

        temp->next = newNode;
    }
}

void print()
{
    int i;

```

```

for(i = 0; i < size; i++)
{
    struct node *temp = chain[i];
    printf("chain[%d]-->",i);
    while(temp)
    {
        printf("%d -->",temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
}

int main()
{
    //init array of list to NULL
    init();

    insert(7);
    insert(0);
    insert(3);
    insert(10);
    insert(4);
    insert(5);

    print();

    return 0;
}

```

Dynamic Array using linked list

A dynamic array is a data structure that can resize itself automatically when elements are added or removed. Unlike regular (static) arrays in languages like C or Java, which have a fixed size defined at compile time, a dynamic array grows or shrinks at runtime as needed.

Key Features of Dynamic Arrays:

Feature	Description
Resizable	Automatically expands or contracts as data is added or removed
Contiguous memory	Like static arrays, elements are stored in a contiguous block of memory
Random Access	Can access any element directly using an index, just like normal arrays
Efficient Append	Appending elements is fast (amortized $O(1)$ time)
Memory Reallocation	When capacity is full, a new larger array is created and old elements are copied over

How Dynamic Arrays Work (Internally):

1. Start with a small fixed-size array (say size 4).
2. As elements are added:
 - If the array is full, **double the size** ($4 \rightarrow 8 \rightarrow 16 \dots$).
 - Allocate a new larger array.
 - **Copy all old elements** to the new array.
 - Delete the old array.
3. This process ensures that insertion at the end is still efficient.

Dynamic Array vs. Linked List:

Feature	Dynamic Array	Linked List
Access by index	Fast ($O(1)$)	Slow ($O(n)$)
Insertion (middle)	Slow ($O(n)$)	Fast ($O(1)$ if position known)
Memory usage	Less (contiguous block)	More (extra pointer per node)
Resizing	Needs reallocation	Grows dynamically

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define the structure for a node
```

```
typedef struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
} Node;
```

```
// Function to create a new node
```

```
Node* createNode(int value) {
```

```
    Node* newNode = (Node*)malloc(sizeof(Node));
```

```
    if (!newNode) {
```

```
        printf("Memory error\n");
```

```
        exit(1);
```

```
    }
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
    return newNode;
```

```
}
```

```
// Function to append an element at the end
```

```
void append(Node** headRef, int value) {
```

```
    Node* newNode = createNode(value);
```

```

if (*headRef == NULL) {
    *headRef = newNode;
    return;
}

Node* temp = *headRef;
while (temp->next != NULL) {
    temp = temp->next;
}
temp->next = newNode;
}

// Function to display the list (dynamic array)
void display(Node* head) {
    printf("Dynamic Array: ");
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

// Function to delete an element at a specific index
void deleteAtIndex(Node** headRef, int index) {
    Node* temp = *headRef;
    if (index == 0 && temp != NULL) {
        *headRef = temp->next;
        free(temp);
        return;
    }

    for (int i = 0; temp != NULL && i < index - 1; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) {
        printf("Invalid index\n");
        return;
    }

    Node* nextNode = temp->next->next;
    free(temp->next);
    temp->next = nextNode;
}

```

```
// Main function
int main() {
    Node* dynamicArray = NULL;

    append(&dynamicArray, 10);
    append(&dynamicArray, 20);
    append(&dynamicArray, 30);
    append(&dynamicArray, 40);

    display(dynamicArray);

    deleteAtIndex(&dynamicArray, 2); // Delete 3rd element (index 2)
    display(dynamicArray);

    return 0;
}
```